

DELEGUES, EVENEMENTS ET EXPRESSIONS LAMBDA

Dans ce chapitre, nous allons aborder les délégués, les événements et les expressions lambdas. Les délégués et les événements sont des types du framework .NET que nous n'avons pas encore vus. Ils permettent d'adresser des solutions notamment dans le cadre d'une programmation par événements, comme c'est le cas lorsque nous réalisons des applications nécessitant de réagir à une action faite par un utilisateur. Nous verrons dans ce chapitre que les expressions lambdas vont de pair avec les délégués.

Les délégués (delegate)

Les délégués (en anglais *delegate*) en C# ne s'occupent pas de la classe, ni du personnel... Ils permettent de créer des variables spéciales. Ce sont des variables qui « pointent » vers une méthode. C'est un peu comme les pointeurs de fonctions en C ou C++, sauf qu'ici on sait exactement ce que l'on utilise, car le C# est un langage fortement typé.

Le délégué va nous permettre de définir une signature de méthode et avec lui, nous pourrons pointer vers n'importe quelle méthode qui respecte cette signature.

En général, on utilise un délégué quand on veut passer une méthode en paramètres d'une autre méthode. Un petit exemple sera sans doute plus parlant qu'un long discours. Ainsi, le code suivant :

```
1public class TrieurDeTableau
2{
3    private delegate void DelegateTri(int[] tableau);
4}
```

crée un délégué privé à la classe `TrieurDeTableau` qui permettra de pointer vers des méthodes qui ne retournent rien (`void`) et qui acceptent un tableau d'entier en paramètres.

C'est justement le cas des méthodes `TriAscendant()` et `TriDescendant()` que nous allons rajouter à la classe (ça tombe bien !) :

```
1public class TrieurDeTableau
2{
3    private delegate void DelegateTri(int[] tableau);
4
5    private void TriAscendant(int[] tableau)
6    {
7        Array.Sort(tableau);
8    }
9
10    private void TriDescendant(int[] tableau)
11    {
12        Array.Sort(tableau);
13        Array.Reverse(tableau);
14    }
15}
```

Vous aurez compris que la méthode `TriAscendant` utilise la méthode `Array.Sort` pour trier un tableau par ordre croissant. Inversement, la méthode `TriDescendant()` trie le tableau par ordre décroissant en triant par ordre croissant et en inversant le tableau ensuite.

Il ne reste plus qu'à créer une méthode dans la classe permettant d'utiliser le tri ascendant et le tri descendant, grâce à notre délégué :

```

1public class TrieurDeTableau
2{
3    [...Code supprimé pour plus de clarté...]
4
5    public void DemoTri(int[] tableau)
6    {
7        DelegateTri tri = TriAscendant;
8        tri(tableau);
9        foreach (int i in tableau)
10       {
11           Console.WriteLine(i);
12       }
13
14       Console.WriteLine();
15       tri = TriDescendant;
16       tri(tableau);
17       foreach (int i in tableau)
18       {
19           Console.WriteLine(i);
20       }
21   }
22}

```

Nous voyons ici que dans la méthode `DemoTri` nous commençons par déclarer une variable du type du délégué `DelegateTri`, qui est le délégué que nous avons créé. Puis nous faisons pointer cette variable vers la méthode `TriAscendant()`.

Nul besoin ici d'utiliser les parenthèses, mais juste le nom de la méthode. Il s'agit seulement d'une affectation.

Nous invoquons ensuite la méthode `TriAscendant()` à travers la variable qui va permettre de trier le tableau par ordre croissant avant d'afficher son contenu. Cette fois-ci, il faut bien sûr utiliser les parenthèses car nous invoquons la méthode.

Puis nous faisons pointer la variable vers la méthode `TriDescendant()` qui va nous permettre de faire la même chose mais avec un tri décroissant.

Nous pouvons appeler cette classe de cette façon :

```

1static void Main(string[] args)
2{
3    int[] tableau = new int[] { 4, 1, 6, 10, 8, 5 };
4    new TrieurDeTableau().DemoTri(tableau);
5}

```

Notre code affichera au final les entiers triés par ordre croissant, puis les mêmes entiers triés par ordre décroissant.

Ok, mais pourquoi utiliser ce délégué ? On pourrait très bien appeler d'abord la méthode de tri ascendant et ensuite la méthode de tri descendant. Comme on l'a toujours fait !

Eh bien, l'intérêt ici est que le délégué est très souple et va permettre de réorganiser le code (on parle également de **refactoriser du code**). Ainsi, en rajoutant la méthode suivante dans la classe :

```

1private void TrierEtAfficher(int[] tableau, DelegateTri methodeDeTri)
2{
3    methodeDeTri(tableau);
4    foreach (int i in tableau)
5    {
6        Console.WriteLine(i);
7    }
8}

```

Nous pourrions grandement simplifier la méthode `DemoTri` :

```
1public void DemoTri(int[] tableau)
2{
3    TrierEtAfficher(tableau, TriAscendant);
4    Console.WriteLine();
5    TrierEtAfficher(tableau, TriDescendant);
6}
```

Ce qui produira le même résultat que précédemment. Qu'avons-nous fait ici ?

Nous avons utilisé le délégué comme paramètre d'une méthode. Ce délégué est ensuite utilisé pour invoquer une méthode que nous aurons passée en paramètres. C'est ce que nous faisons en disant d'utiliser la méthode `TrierEtAfficher` une première fois avec la méthode `TriAscendant()` et une deuxième fois avec la méthode `TriDescendant()`.

Plutôt pas mal non ?

Il est même possible de définir la méthode qui sera utilisée à l'intérieur de `TrierEtAfficher()` sans avoir à l'écrire complètement dans le corps de la classe.

Cela peut être utile si la méthode est vouée à n'être utilisée que dans cette unique situation et qu'elle n'est jamais appelée à un autre endroit.

Par exemple, plutôt que de définir complètement la méthode `TriAscendant()`, je pourrais la définir directement au moment de l'appel de la méthode :

```
1public class TrieurDeTableau
2{
3    private delegate void DelegateTri(int[] tableau);
4
5    private void TrierEtAfficher(int[] tableau, DelegateTri methodeDeTri)
6    {
7        methodeDeTri(tableau);
8        foreach (int i in tableau)
9        {
10            Console.WriteLine(i);
11        }
12    }
13
14    public void DemoTri(int[] tableau)
15    {
16        TrierEtAfficher(tableau, delegate(int[] leTableau)
17        {
18            Array.Sort(leTableau);
19        });
20
21        Console.WriteLine();
22
23        TrierEtAfficher(tableau, delegate(int[] leTableau)
24        {
25            Array.Sort(leTableau);
26            Array.Reverse(leTableau);
27        });
28    }
29 }
30}
```

Ainsi, je n'aurai plus besoin de la méthode `TriAscendant()` ni de la méthode `TriDescendant()`.

Le fait de définir la méthode directement au niveau du paramètre d'appel est ce qu'on appelle « **utiliser une méthode anonyme** ». Anonyme car la méthode n'a pas de nom. Elle n'a de vie qu'à cet endroit-là. La syntaxe est un peu particulière, mais au lieu d'utiliser une variable de type `delegate` qui pointe vers une méthode, c'est comme si on écrivait directement la méthode.

On utilise le mot-clé `delegate` suivi de la déclaration du paramètre. Évidemment, le délégué anonyme

doit respecter la signature de `DelegateTri` que nous avons définie plus haut. Enfin, nous faisons suivre avec un bloc de code qui correspond au corps de la méthode anonyme.

À noter que le fait d'utiliser le mot-clé `delegate` revient en fait à créer une classe qui dérive de `System.Delegate` et qui implémente la logique de base d'un délégué. Le C# nous masque tout ceci afin d'être au maximum efficace.

Diffusion multiple, le Multicast

Il faut également savoir que le délégué peut être **multicast**, c'est à dire qu'il peut pointer vers plusieurs méthodes.

Améliorons le premier exemple :

```
1public class TrieurDeTableau
2{
3    private delegate void DelegateTri(int[] tableau);
4
5    private void TriAscendant(int[] tableau)
6    {
7        Array.Sort(tableau);
8        foreach (int i in tableau)
9        {
10            Console.WriteLine(i);
11        }
12        Console.WriteLine();
13    }
14
15    private void TriDescendant(int[] tableau)
16    {
17        Array.Sort(tableau);
18        Array.Reverse(tableau);
19        foreach (int i in tableau)
20        {
21            Console.WriteLine(i);
22        }
23    }
24
25    public void DemoTri(int[] tableau)
26    {
27        DelegateTri tri = TriAscendant;
28        tri += TriDescendant;
29        tri(tableau);
30    }
31}
```

Ici, j'utilise `Console.WriteLine` directement dans chaque méthode de tri afin de bien voir le résultat du tri du tableau.

L'important est de voir que dans la méthode `DemoTri`, je commence par créer un délégué que je fais pointer vers la méthode `TriAscendant`. Puis j'ajoute à ce délégué, avec l'opérateur `+=`, une nouvelle méthode, à savoir `TriDescendant`.

Désormais, le fait d'invoquer le délégué va invoquer en fait les deux méthodes. Ce qui produira en sortie :

```
1
4
5
6
8
10

10
8
6
```

5
4
1

Ce détail prend toute son importance avec les événements que nous verrons plus loin.

À noter que le résultat de ce code est évidemment identique en utilisant les méthodes anonymes :

```
1public void DemoTri(int[] tableau)
2{
3    DelegateTri tri = delegate(int[] leTableau)
4    {
5        Array.Sort(leTableau);
6        foreach (int i in tableau)
7        {
8            Console.WriteLine(i);
9        }
10       Console.WriteLine();
11    };
12    tri += delegate(int[] leTableau)
13    {
14        Array.Sort(tableau);
15        Array.Reverse(tableau);
16        foreach (int i in tableau)
17        {
18            Console.WriteLine(i);
19        }
20    };
21    tri(tableau);
22}
```

Il faut quand même remarquer que l'ordre dans lequel sont appelées les méthodes n'est pas garanti et ne dépend pas forcément de l'ordre dans lequel nous les avons ajoutées au délégué.

Les délégués génériques Action et Func

C'est très bien tout ça, mais cela veut dire qu'à chaque fois que je vais avoir besoin d'utiliser un délégué, je vais devoir créer un nouveau type en utilisant le mot-clé `delegate` ?

Pas forcément, c'est là qu'interviennent les délégués génériques `Action` et `Func`. `Action` est un délégué qui permet de pointer vers une méthode qui ne renvoie rien et qui peut accepter jusqu'à 16 types différents. Cela veut dire que le code précédent peut être remplacé par :

```
1public class TrieurDeTableau
2{
3    private void TrierEtAfficher(int[] tableau, Action<int[]> methodeDeTri)
4    {
5        methodeDeTri(tableau);
6        foreach (int i in tableau)
7        {
8            Console.WriteLine(i);
9        }
10    }
11
12    public void DemoTri(int[] tableau)
13    {
14        TrierEtAfficher(tableau, delegate(int[] leTableau)
15        {
16            Array.Sort(leTableau);
17        });
18
19        Console.WriteLine();
20
21        TrierEtAfficher(tableau, delegate(int[] leTableau)
```

```

22     {
23         Array.Sort(tableau);
24         Array.Reverse(tableau);
25     });
26 }
27}

```

Notez que la différence se situe au niveau du paramètre de la méthode `TrierEtAfficher` qui prend un `Action<int[]>`. En fait, cela est équivalent à créer un délégué qui ne renvoie rien et qui prend un tableau d'entier en paramètre.

Si notre méthode avait deux paramètres, il aurait suffi d'utiliser la forme de `Action` avec plusieurs paramètres génériques, par exemple `Action<int[], string>` pour avoir une méthode qui ne renvoie rien et qui prend un tableau d'entier et une chaîne de caractères en paramètres.

Lorsque la méthode renvoie quelque chose, on peut utiliser le délégué `Func<T>`, sachant qu'ici, c'est le dernier paramètre générique qui sera du type de retour du délégué. Par exemple :

```

1public class Operations
2{
3    public void DemoOperations()
4    {
5        double division = Calcul(delegate(int a, int b)
6        {
7            return (double)a / (double)b;
8        }, 4, 5);
9
10       double puissance = Calcul(delegate(int a, int b)
11       {
12           return Math.Pow((double)a, (double)b);
13       }, 4, 5);
14
15       Console.WriteLine("Division : " + division);
16       Console.WriteLine("Puissance : " + puissance);
17   }
18
19   private double Calcul(Func<int, int, double> methodeDeCalcul, int a, int b)
20   {
21       return methodeDeCalcul(a, b);
22   }
23}

```

Ici, dans la méthode `Calcul`, on utilise le délégué `Func` pour indiquer que la méthode prend deux entiers en paramètres et renvoie un double.

Si nous utilisons cette classe avec le code suivant :

```

1class Program
2{
3    static void Main(string[] args)
4    {
5        new Operations().DemoOperations();
6    }
7}

```

Nous aurons :

```

Division : 0,8
Puissance : 1024

```

Les expressions lambdas

Non, il ne s'agit pas d'une expression qui danse la lambada, mais d'une façon simplifiée d'écrire les délégués que nous avons vus au-dessus. 🤘

Ainsi, le code suivant :

```
1DelegateTri tri = delegate(int[] leTableau)
2{
3    Array.Sort(leTableau);
4};
```

peut également s'écrire de cette façon :

```
1DelegateTri tri = (leTableau) =>
2{
3    Array.Sort(leTableau);
4};
```

Cette syntaxe est particulière. La variable `leTableau` permet de spécifier le paramètre d'entrée de l'expression lambda. Ce paramètre est écrit entre parenthèses. Ici, pas besoin d'indiquer son type vu qu'il est connu par la signature associée au délégué. On utilise ensuite la flèche « `=>` » pour définir l'expression lambda qui sera utilisée. Elle s'écrit de la même façon qu'une méthode, dans un bloc de code.

L'expression lambda « `(leTableau) =>` » se lit : « `leTableau` conduit à ».

Dans le corps de la méthode, nous voyons que nous utilisons la variable `leTableau` de la même façon que précédemment.

Dans ce cas précis, il est encore possible de raccourcir l'écriture car la méthode ne contient qu'une seule ligne, on pourra alors l'écrire de cette façon :

```
1TrierEtAfficher(tableau, leTableau => Array.Sort(leTableau));
```

S'il n'y a qu'un seul paramètre à l'expression lambda, on peut omettre les parenthèses.

Quand il y a deux paramètres, on les sépare par une virgule. À noter qu'on n'indique nulle part le type de retour s'il y en a un.

Notre expression lambda remplaçant le calcul de la division peut donc s'écrire ainsi :

```
1double division = Calcul((a, b) =>
2{
3    return (double)a / (double)b;
4}, 4, 5);
```

Lorsque l'instruction possède une unique ligne, on peut encore simplifier l'écriture, ce qui donne :

```
1double division = Calcul((a, b) => (double)a / (double)b, 4, 5);
```

Pourquoi tout ce blabla sur les `delegate` et les expressions lambdas ?

Pour deux raisons :

- parce que les délégués sont la base des événements ;
- à cause des méthodes d'extensions LINQ.

Nous parlerons dans la partie suivante des méthodes d'extensions LINQ. Quant aux événements, explorons-les immédiatement !

Les événements

Les événements sont un mécanisme du C# permettant à une classe d'être notifiée d'un changement. Par exemple, on peut vouloir s'abonner à un changement de prix d'une voiture.

La base des événements est le délégué. On pourra stocker dans un événement un ou plusieurs délégués qui pointent vers des méthodes respectant la signature de l'événement.

Un événement est défini grâce au mot-clé `event`. Prenons cet exemple :

```
1public class Voiture
2{
3    public delegate void DelegateDeChangementDePrix(decimal nouveauPrix);
4    public event DelegateDeChangementDePrix ChangementDePrix;
5    public decimal Prix { get; set; }
6
7    public void PromoSurLePrix()
8    {
9        Prix = Prix / 2;
10       if (ChangementDePrix != null)
11           ChangementDePrix(Prix);
12    }
13}
```

Dans la classe `Voiture`, nous définissons un délégué qui ne retourne rien et qui prend en paramètre un décimal. Nous définissons ensuite un événement basé sur ce délégué, avec comme nous l'avons vu, l'utilisation du mot-clé `event`. Enfin, dans la méthode de promotion, après un changement de prix (division par 2), nous notifions les éventuels objets qui se seraient abonnés à cet événement en invoquant l'événement et en lui fournissant en paramètre le nouveau prix.

À noter que nous testons d'abord s'il y a un abonné à l'événement (en testant s'il est différent de `null`) avant de le lever.

Pour s'abonner à cet événement, il suffit d'utiliser le code suivant :

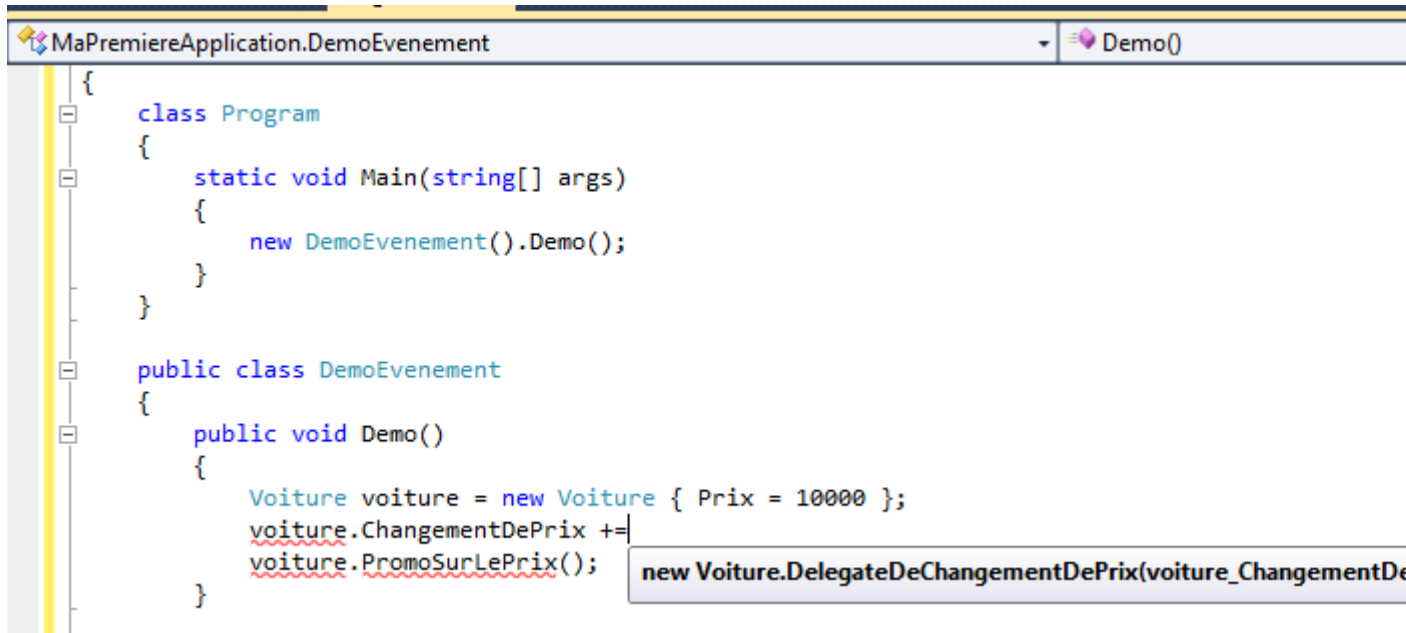
```
1class Program
2{
3    static void Main(string[] args)
4    {
5        new DemoEvenement().Demo();
6    }
7}
8
9public class DemoEvenement
10{
11    public void Demo()
12    {
13        Voiture voiture = new Voiture { Prix = 10000 };
14
15        Voiture.DelegateDeChangementDePrix delegateChangementDePrix = voiture_ChangementDePrix;
16        voiture.ChangementDePrix += delegateChangementDePrix;
17
18        voiture.PromoSurLePrix();
19    }
20
21    private void voiture_ChangementDePrix(decimal nouveauPrix)
22    {
23        Console.WriteLine("Le nouveau prix est de : " + nouveauPrix);
24    }
25}
```

Nous créons une voiture, et nous créons un délégué du même type que l'événement. Nous le faisons pointer vers une méthode qui respecte la signature du délégué. Ainsi, à chaque changement de prix, la méthode `voiture_ChangementDePrix` va être appelée et le paramètre `nouveauPrix` possèdera le nouveau prix qui vient d'être calculé.

Appelons la promotion en invoquant la méthode `ChangementDePrix()`, nous pouvons nous rendre compte que l'application nous affiche le nouveau prix qui est l'ancien divisé par 2.

Lorsque nous commençons à écrire le code qui va permettre de nous abonner à l'événement, la complétion automatique nous propose facilement de créer une méthode qui respecte la signature de l'événement. Il

suffit de taper l'événement de rajouter un += et il nous propose d'insérer tout automatiquement si nous appuyons sur la tabulation :



Ce qui génère le code suivant :

```
1voiture.ChangementDePrix += new Voiture.DelegateDeChangeementDePrix(voiture_ChangementDePrix);
```

ainsi que la méthode :

```
1void voiture_ChangementDePrix(decimal nouveauPrix)
2{
3    throw new NotImplementedException();
4}
```

On peut aisément simplifier l'abonnement avec :

```
1voiture.ChangementDePrix += voiture_ChangementDePrix;
```

comme on l'a déjà vu. Notez que vous pouvez également rajouter la visibilité `private` sur la méthode générée afin que cela soit plus explicite.

```
1private void voiture_ChangementDePrix(decimal nouveauPrix)
2{
3}
```

L'utilisation du « += » permet d'ajouter un nouveau délégué à l'événement. Il sera éventuellement possible d'ajouter un autre délégué avec le même opérateur, ainsi deux méthodes seront désormais notifiées en cas de changement de prix. Inversement, il est possible de se désabonner d'un événement en utilisant l'opérateur « -= ».

Les événements sont beaucoup utilisés dans les applications en C#, comme les applications clients lourds développées avec WPF par exemple. Ce sont des applications comme un traitement de texte ou un navigateur internet. Par exemple, lorsque l'on clique sur un bouton, un événement est levé.

Ces événements utilisent en général une construction à base du délégué `EventHandler` ou sa version générique `EventHandler<>`. Ce délégué accepte deux paramètres. Le premier de type `object` qui

représente la source de l'événement, c'est-à-dire l'objet qui a levé l'événement. Le second est une classe qui dérive de la classe de base EventArgs.

Réécrivons notre exemple avec ce nouveau handler. Nous avons donc besoin en premier lieu d'une classe qui dérive de la classe EventArgs :

```
1public class ChangementDePrixEventArgs : EventArgs
2{
3    public decimal Prix { get; set; }
4}
```

Plus besoin de déclaration de délégué, nous utilisons directement EventHandler dans notre classe Voiture :

```
1public class Voiture
2{
3    public event EventHandler<ChangementDePrixEventArgs> ChangementDePrix;
4    public decimal Prix { get; set; }
5
6    public void PromoSurLePrix()
7    {
8        Prix = Prix / 2;
9        if (ChangementDePrix != null)
10            ChangementDePrix(this, new ChangementDePrixEventArgs { Prix = Prix });
11    }
12}
```

Et notre démo devient :

```
1class Program
2{
3    static void Main(string[] args)
4    {
5        new DemoEvenement().Demo();
6    }
7}
8
9public class DemoEvenement
10{
11    public void Demo()
12    {
13        Voiture voiture = new Voiture { Prix = 10000 };
14        voiture.ChangementDePrix += voiture_ChangementDePrix;
15        voiture.PromoSurLePrix();
16    }
17
18    private void voiture_ChangementDePrix(object sender, ChangementDePrixEventArgs e)
19    {
20        Console.WriteLine("Le nouveau prix est de : " + e.Prix);
21    }
22}
23
24}
```

Remarquons la méthode voiture_ChangementDePrix qui prend désormais deux paramètres. Le premier représente l'objet Voiture, si nous en avons besoin, nous pourrions l'utiliser avec un cast adéquat. Le second représente l'objet contenant le prix de la voiture en promotion.

À noter qu'en général, nous allons beaucoup utiliser les événements définis par le framework .NET. Il est cependant assez rare d'avoir à en définir un soi-même.

En résumé

- Les délégués permettent de créer des variables pointant vers des méthodes.
- Les délégués sont à la base des événements.
- On utilise les expressions lambdas pour simplifier l'écriture des délégués.
- Les événements sont un mécanisme du C# permettant à une classe d'être notifiée d'un changement.